Practical No: 1(Data Pre-processing and Exploration)

a. Load a CSV dataset. Handle missing values, inconsistent formatting, and outliers.

b. Load a dataset, calculate descriptive summary statistics, create visualizations using different graphs, and identify potential features and target variables

c. Create or Explore datasets to use all pre-processing routines like label encoding, scaling.

Practical No: 3

Simple Linear Regression

A. Fit a linear regression model on a dataset. Interpret coefficients, make predictions, and evaluate performance using metrics like R-squared and MSE

PRACTICAL 4

Aim: Perform binary classification using logistic regression. Calculate accuracy, precision, recall, and understand the ROC curve.

PRACTICAL 5

Aim: Implement a Support Vector Machine for any relevant dataset.

PRACTICAL 6

Aim: To implement the Naive Bayes classification algorithm using Python to predict whether a person should play based on weather conditions and temperature, and evaluate the model's accuracy.

PRACTICAL 7

Aim: Implement and demonstrate k-nearest Neighbor algorithm. Read the training data from a

.CSV file and build the model to classify a test sample. Print both correct and wrong predictions.

Practical No: 1

Data Pre-processing and Exploration

a. Load a CSV dataset. Handle missing values, inconsistent formatting, and outliers.

b. Load a dataset, calculate descriptive summary statistics, create visualizations using different graphs, and identify potential features and target variables

c. Create or Explore datasets to use all pre-processing routines like label encoding, scaling.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
df = pd.read_csv("Churn_Modelling.csv")
df.shape
df.head()
df.tail()
df.info()
df.isnull().sum()
df.dropna()
df.isnull().sum()
df.duplicated().sum()
df.drop_duplicates()
df.duplicated().sum()
df.columns.str.strip()
df["Surname"] = df["Surname"].str.strip()
```

```
df["Gender"] = df["Gender"].str.strip()
df["Geography"] = df["Geography"].str.strip()
le = LabelEncoder()
df["Gender"] = le.fit_transform(df["Gender"])
df["Geography"] = le.fit_transform(df["Geography"])
df = df.drop(["RowNumber", "CustomerId", "Surname"], axis=1)
df.describe()
df.corr()
plt.figure(figsize=(8,5))
plt.hist(df["Age"], bins=20, color="skyblue", edgecolor="black")
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
plt.figure(figsize=(7,5))
sns.barplot(x="Gender", y="Balance", data=df)
plt.title("Gender vs Average Balance")
plt.show()
plt.figure(figsize=(7,5))
plt.scatter(df["Age"], df["Balance"], color="blue")
plt.xlabel("Age")
plt.ylabel("Balance")
plt.title("Age vs Balance")
plt.show()
scaler = StandardScaler()
X=df.drop(“Exit”,axis=1)
X_scaled = scaler.fit_transform(X)
print("\nScaled Features")
print(X_scaled)
```

Practical No: 3

Simple Linear Regression

A. Fit a linear regression model on a dataset. Interpret coefficients, make predictions, and evaluate performance using metrics like R-squared and MSE

Code:

```
import pandas as pd

import numpy as np

from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score


housing = fetch_california_housing()


df = pd.DataFrame(housing.data, columns=housing.feature_names)

df["Price"] = housing.target


print("Dataset:")

print(df.head())
```

```
Dataset:
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude  Price
0    -122.23  4.526
1    -122.22  3.585
2    -122.24  3.521
3    -122.25  3.413
4    -122.25  3.422
```

```
X = df.drop("Price", axis=1)
```

y = df["Price"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
14196	3.2596	33.0	5.017657	1.006421	2300.0	3.691814	32.71	-117.03
8267	3.8125	49.0	4.473545	1.041005	1314.0	1.738095	33.77	-118.16
17445	4.1563	4.0	5.645833	0.985119	915.0	2.723214	34.66	-120.48
14265	1.9425	36.0	4.002817	1.033803	1418.0	3.994366	32.69	-117.11
2271	3.5542	43.0	6.268421	1.134211	874.0	2.300000	36.78	-119.80
...	...	...	...	...	...	...	...	...
11284	6.3700	35.0	6.129032	0.926267	658.0	3.032258	33.78	-117.96
11964	3.0500	33.0	6.868597	1.269488	1753.0	3.904232	34.02	-117.43
5390	2.9344	36.0	3.986717	1.079696	1756.0	3.332068	34.03	-118.38
860	5.7192	15.0	6.395349	1.067979	1777.0	3.178891	37.58	-121.96
15795	2.5755	52.0	3.402576	1.058776	2619.0	2.108696	37.77	-122.42

16512 rows × 8 columns

model = LinearRegression()

model.fit(X_train, y_train)

```
model = LinearRegression()

model.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

y_pred = model.predict(X_test)

print("\nIntercept:")

print(model.intercept_)

```
y_pred = model.predict(X_test)

print("\nIntercept:")
print(model.intercept_)

y_pred
```

```
Intercept:
-37.02327770606409
array([0.71912284, 1.76401657, 2.70965883, ..., 4.46877017, 1.18751119,
       2.00940251])
```

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

y_pred

print("\nMean Squared Error (MSE):", mse)

print("R-squared Score:", r2)

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nMean Squared Error (MSE):", mse)
print("R-squared Score:", r2)
```

```
Mean Squared Error (MSE): 0.5558915986952444
R-squared Score: 0.5757877060324508
```

PRACTICAL 4

Aim: Perform binary classification using logistic regression. Calculate accuracy, precision, recall,

and understand the ROC curve.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_curve, roc_auc_score

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)

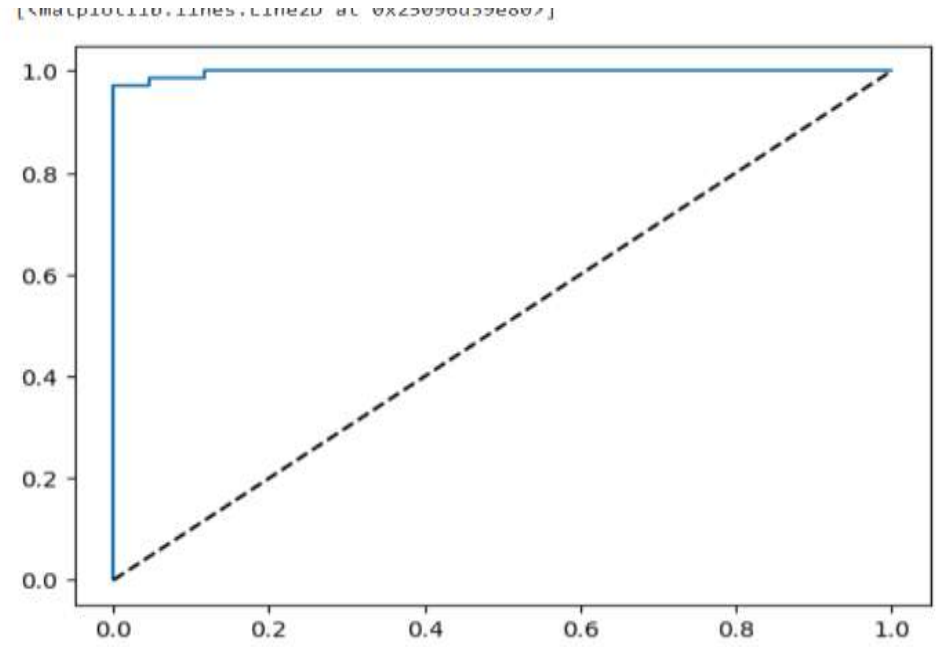
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
print("Precision:", round(precision_score(y_test, y_pred), 3))
print("Recall:", round(recall_score(y_test, y_pred), 3))
```

```
Accuracy: 0.956
Precision: 0.946
Recall: 0.986
```

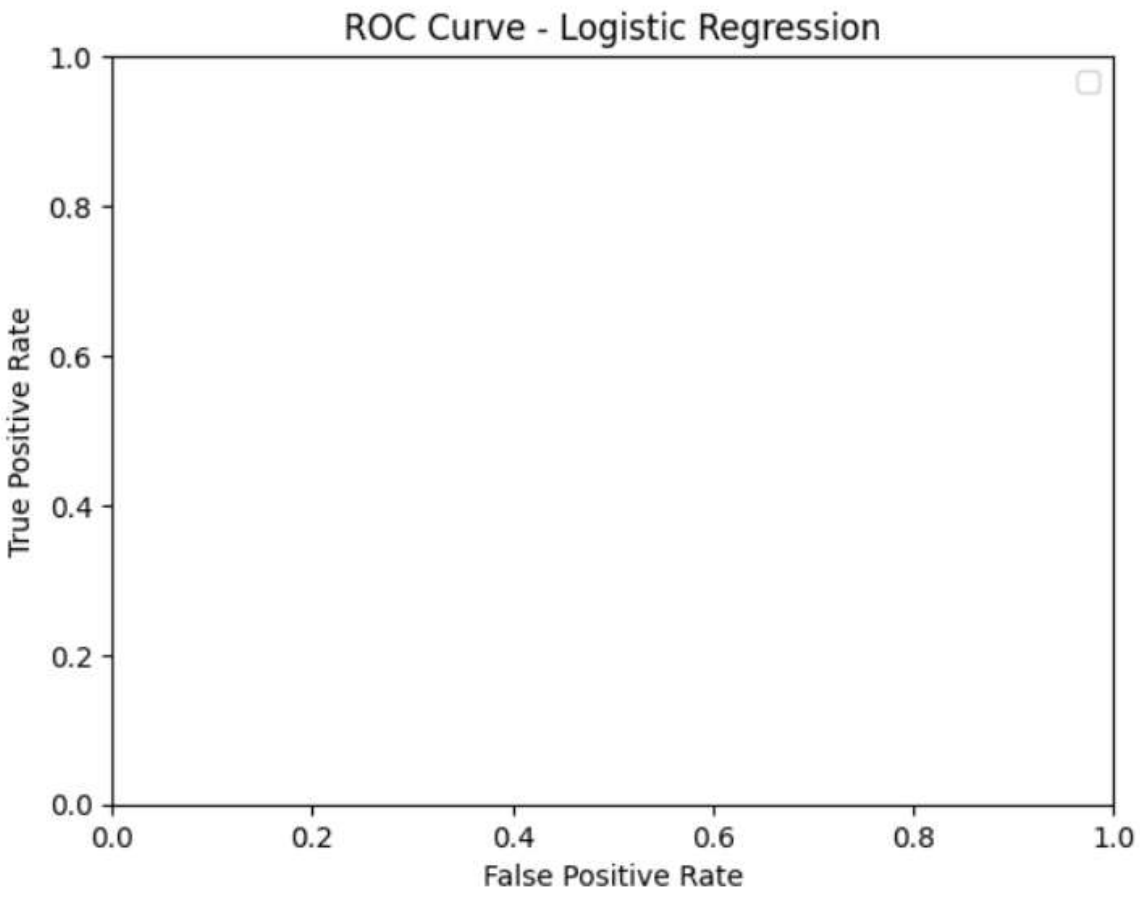
```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)

plt.plot(fpr, tpr, label="ROC Curve (AUC = %.2f)" % auc)
plt.plot([0, 1], [0, 1], "k--", label="Random Classifier")
```




```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Logistic Regression")
plt.legend()
plt.show()

print("AUC Score:", round(auc, 3))
```



AUC Score: 0.998

PRACTICAL 5

Aim: Implement a Support Vector Machine for any relevant dataset.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train SVM model
model = SVC(kernel='linear')
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Display results
print("Actual Values:", y_test)
print("Predicted Values:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred, target_names=data.target_names))

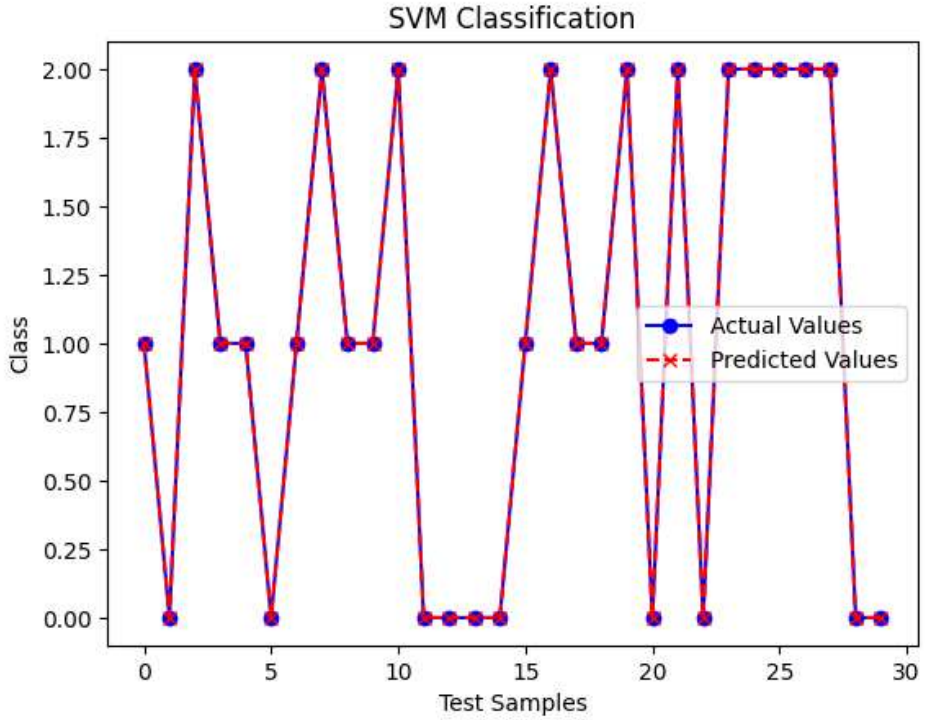

# Plot actual vs predicted values
plt.plot(y_test, 'bo-', label='Actual Values')
plt.plot(y_pred, 'rx--', label='Predicted Values')
plt.xlabel("Test Samples")
plt.ylabel("Class")
plt.title("SVM Classification")
plt.legend()
plt.show()
```

```
Actual Values: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Predicted Values: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```



PRACTICAL 6

Aim: To implement the Naive Bayes classification algorithm using Python to predict whether a person should play based on weather conditions and temperature, and evaluate the model's accuracy.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Create sample dataset
data = {
    'Weather': ['Sunny', 'Sunny', 'Rainy', 'Overcast', 'Rainy',
                'Sunny', 'Overcast', 'Rainy', 'Sunny', 'Overcast'],
    'Temperature': ['Hot', 'Hot', 'Cold', 'Mild', 'Cold',
                    'Mild', 'Hot', 'Mild', 'Cold', 'Mild'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes',
             'Yes', 'Yes', 'No', 'Yes', 'Yes']
}

df = pd.DataFrame(data)

# Convert categorical data into numerical values
df['Weather'] = df['Weather'].map(
    {'Sunny': 0, 'Overcast': 1, 'Rainy': 2}
)
df['Temperature'] = df['Temperature'].map(
    {'Hot': 0, 'Mild': 1, 'Cold': 2}
)
```

```
df['Play'] = df['Play'].map({'No': 0, 'Yes': 1})

# Separate input and output
X = df[['Weather', 'Temperature']]
y = df['Play']

# Train the Naive Bayes model
model = GaussianNB()
model.fit(X, y)

# Test a new sample: Sunny weather and Mild temperature
test_sample = pd.DataFrame({
    'Weather': [0],
    'Temperature': [1]
})

prediction = model.predict(test_sample)

# Display result
if prediction[0] == 1:
    print("Prediction: Yes, Play")
else:
    print("Prediction: No, Don't Play")

# Calculate accuracy on sample dataset
y_pred = model.predict(X)
print("Accuracy:", accuracy_score(y, y_pred))
```

```
Prediction: Yes, Play
Accuracy: 0.8
```

PRACTICAL 7

Aim: Implement and demonstrate k-nearest Neighbor algorithm. Read the training data from a

.CSV file and build the model to classify a test sample. Print both correct and wrong

predictions.

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Create sample dataset and save it as a CSV file
data = {
    'Height': [150, 160, 170, 180, 155, 165, 175, 185, 152, 178],
    'Weight': [50, 55, 65, 80, 52, 60, 75, 85, 48, 78],
    'Class': ['A', 'A', 'B', 'B', 'A', 'A', 'B', 'B', 'A', 'B']
}

df = pd.DataFrame(data)
df.to_csv('data.csv', index=False)

# Read training data from CSV file
df = pd.read_csv('data.csv')

X = df[['Height', 'Weight']]
y = df['Class']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# Build KNN model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Predict test samples
y_pred = model.predict(X_test)

# Print correct and wrong predictions
print("Actual\tPredicted\tResult")

for actual, predicted in zip(y_test, y_pred):
    if actual == predicted:
        print(actual, "\t", predicted, "\t\tCorrect")
    else:
        print(actual, "\t", predicted, "\t\tWrong")

# Print accuracy
print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

```
Actual  Predicted       Result
A        A              Correct
A        A              Correct
A        B              Wrong

Accuracy: 0.6666666666666666
```